

OKD 4.19 UPI Deployment Guide

CentOS Stream CoreOS 9 (SCOS 9) | VMware ESXi 8 Console | No vCenter

OKD Version: `4.19.0-okd-scos.3` (latest stable 4.19) **Node OS:** CentOS Stream CoreOS 9 (SCOS 9) **Platform:** VMware ESXi 8 direct console (no vCenter) **Bastion OS:** Rocky Linux 9 **Install Method:** UPI (User Provisioned Infrastructure) — manual `coreos-installer`
Domain: `svcexpert.net` | **Cluster:** `ocp`

Why OKD 4.19? This is the first stable OKD release where SCOS is the native boot image with no FCOS-to-SCOS pivot step. Bare metal and VMware UPI installs work cleanly with `coreos-installer` and no pivot workarounds. The GRUB-loop issue experienced with SCOS 10 ISOs on BIOS/EFI virtual hardware is avoided by using the SCOS 9 stream which has broader VMware BIOS/EFI compatibility.

Network Reference

Hostname	IP	Role
<code>bastion.ocp.svcexpert.net</code>	<code>192.168.100.10</code>	Rocky 9 — HAProxy, HTTPD, installer
<code>bootstrap.ocp.svcexpert.net</code>	<code>192.168.100.14</code>	Bootstrap (temporary, deleted after install)
<code>cp0.ocp.svcexpert.net</code>	<code>192.168.100.15</code>	Control Plane 0
<code>cp1.ocp.svcexpert.net</code>	<code>192.168.100.16</code>	Control Plane 1
<code>cp2.ocp.svcexpert.net</code>	<code>192.168.100.17</code>	Control Plane 2
<code>api.ocp.svcexpert.net</code>	<code>192.168.100.11</code>	API VIP (alias on bastion ens18)
<code>api-int.ocp.svcexpert.net</code>	<code>192.168.100.12</code>	Internal API VIP (alias on bastion ens18)
<code>*.apps.ocp.svcexpert.net</code>	<code>192.168.100.13</code>	Apps wildcard VIP (alias on bastion ens18)

Gateway: `192.168.100.1` | DNS: `192.168.100.251` (Windows Server) | Subnet: `/24`

VM Specifications

VM	vCPU	RAM	Disk	Lifecycle
bootstrap	4	16 GB	120 GB	Delete after cluster forms
cp0	4	16 GB	120 GB	Permanent
cp1	4	16 GB	120 GB	Permanent
cp2	4	16 GB	120 GB	Permanent

Phase 1 — Bastion Setup (Rocky Linux 9)

1.1 Set Hostname

```
bash

hostnamectl set-hostname bastion.ocp.svcexpert.net
exec bash
```

1.2 Configure Multiple IPs on ens18

IPs `.11`, `.12`, `.13` are VIPs added as secondary addresses on the same interface as the bastion's primary IP `.10`.

```
bash
```

```
# Verify interface name first
nmcli device status
# Adjust "ens18" below if your interface has a different name

# Primary IP (set if not already configured)
nmcli con mod "ens18" \
  ipv4.addresses "192.168.100.10/24" \
  ipv4.gateway "192.168.100.1" \
  ipv4.dns "192.168.100.251" \
  ipv4.method manual

# Add VIP secondary addresses
nmcli con mod "ens18" +ipv4.addresses "192.168.100.11/24"
nmcli con mod "ens18" +ipv4.addresses "192.168.100.12/24"
nmcli con mod "ens18" +ipv4.addresses "192.168.100.13/24"

# Apply
nmcli con up "ens18"

# Verify – must show all four IPs
ip addr show ens18 | grep "inet "
```

Expected output:

```
inet 192.168.100.10/24 ...
inet 192.168.100.11/24 ...
inet 192.168.100.12/24 ...
inet 192.168.100.13/24 ...
```

1.3 Install Required Packages

```
bash
```

```
dnf update -y
dnf install -y \
    haproxy \
    httpd \
    bind-utils \
    curl wget tar jq \
    openssl nmap nc \
    firewallld

systemctl enable --now firewallld
```

1.4 Firewall Rules

```
bash

firewall-cmd --permanent --add-port=6443/tcp # Kubernetes API
firewall-cmd --permanent --add-port=22623/tcp # Machine Config Server
firewall-cmd --permanent --add-port=80/tcp    # Ingress HTTP
firewall-cmd --permanent --add-port=443/tcp   # Ingress HTTPS
firewall-cmd --permanent --add-port=8080/tcp  # HTTPD ignition server
firewall-cmd --permanent --add-port=9000/tcp  # HAProxy stats
firewall-cmd --reload
firewall-cmd --list-all
```

1.5 Configure HAProxy

```
bash
```

```
cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.orig
```

```
cat > /etc/haproxy/haproxy.cfg << 'EOF'
```

```
global
```

```
    log          /dev/log local0
    log          /dev/log local1 notice
    chroot       /var/lib/haproxy
    pidfile      /var/run/haproxy.pid
    maxconn      4000
    user         haproxy
    group        haproxy
    daemon
    tune.ssl.default-dh-param 2048
```

```
defaults
```

```
    mode                tcp
    log                 global
    option              tcplog
    option              dontlognull
    option              redispatch
    retries             3
    timeout connect     10s
    timeout client      1m
    timeout server      1m
    maxconn             3000
```

```
#-----
```

```
# HAProxy Stats - http://bastion:9000/stats
```

```
#-----
```

```
listen stats
```

```
    bind *:9000
    mode http
    stats enable
    stats uri /stats
    stats refresh 10s
    stats show-legends
```

```
#-----
```

```
# Kubernetes API - port 6443
```

```
# Bootstrap included initially; comment out after bootstrap removal
```

```
#-----
```

```
frontend k8s-api
```

```
    bind *:6443
```

```
    default_backend k8s-api-backend

backend k8s-api-backend
    balance roundrobin
    option tcp-check
    server bootstrap 192.168.100.14:6443 check
    server cp0       192.168.100.15:6443 check
    server cp1       192.168.100.16:6443 check
    server cp2       192.168.100.17:6443 check

#-----
# Machine Config Server – port 22623
# Bootstrap included initially; comment out after bootstrap removal
#-----
frontend mcs
    bind *:22623
    default_backend mcs-backend

backend mcs-backend
    balance roundrobin
    option tcp-check
    server bootstrap 192.168.100.14:22623 check
    server cp0       192.168.100.15:22623 check
    server cp1       192.168.100.16:22623 check
    server cp2       192.168.100.17:22623 check

#-----
# Ingress HTTP – port 80
# Add worker IPs when workers are deployed
#-----
frontend ingress-http
    bind *:80
    default_backend ingress-http-backend

backend ingress-http-backend
    balance roundrobin
    option tcp-check
    # server worker0 192.168.100.20:80 check
    # server worker1 192.168.100.21:80 check

#-----
# Ingress HTTPS – port 443
#-----
frontend ingress-https
```

```
bind *:443
default_backend ingress-https-backend

backend ingress-https-backend
    balance roundrobin
    option tcp-check
    # server worker0 192.168.100.20:443 check
    # server worker1 192.168.100.21:443 check
EOF

# Allow HAProxy to connect to any port (SELinux)
setsebool -P haproxy_connect_any 1

systemctl enable --now haproxy
systemctl status haproxy --no-pager
```

1.6 Configure HTTPD on Port 8080

```
bash

# Move HTTPD to port 8080 to avoid conflict with ingress port 80
sed -i 's/^Listen 80$/Listen 8080/' /etc/httpd/conf/httpd.conf

# Create ignition directory
mkdir -p /var/www/html/ignition
chmod 755 /var/www/html/ignition

systemctl enable --now httpd
systemctl status httpd --no-pager

# Test
curl -I http://127.0.0.1:8080/
```

1.7 Download OKD 4.19 Binaries

```
bash
```

```
mkdir -p /opt/okd/{bin,install}
cd /opt/okd

# Pinned to latest stable 4.19 point release
OKD_VERSION="4.19.0-okd-scos.3"

# Download openshift-install
wget -O /opt/okd/openshift-install-linux.tar.gz \
  "https://github.com/okd-project/okd/releases/download/4.19.0-okd-scos.3/openshift-install-linux.tar.gz"

# Download oc client
wget -O /opt/okd/openshift-client-linux.tar.gz \
  "https://github.com/okd-project/okd/releases/download/4.19.0-okd-scos.3/openshift-client-linux.tar.gz"

# Extract and install
tar -xzf openshift-install-linux.tar.gz -C /opt/okd/bin/
tar -xzf openshift-client-linux.tar.gz -C /opt/okd/bin/

cp /opt/okd/bin/openshift-install /usr/local/bin/
cp /opt/okd/bin/oc /usr/local/bin/
cp /opt/okd/bin/kubectl /usr/local/bin/

# Verify - must show 4.19.0-okd-scos.3
openshift-install version
oc version --client
```

1.8 Download SCOS 9 Live ISO

Use `openshift-install coreos print-stream-json` to get the exact ISO URL matched to this specific OKD release. Never substitute a manually guessed URL.

```
bash
```

```
# Print the full SCOS stream metadata for this release
openshift-install coreos print-stream-json | jq .

# Extract the live ISO URL for x86_64
SCOS_ISO_URL=$(openshift-install coreos print-stream-json \
  | jq -r '.architectures.x86_64.artifacts.metal.formats.iso.disk.location')

echo "SCOS 9 ISO URL: ${SCOS_ISO_URL}"

# Download to the ignition web directory
wget -O /var/www/html/ignition/scos9.iso "${SCOS_ISO_URL}"
chmod 644 /var/www/html/ignition/scos9.iso

# Verify it is a real bootable ISO (must report ISO 9660 ... bootable)
file /var/www/html/ignition/scos9.iso
ls -lh /var/www/html/ignition/scos9.iso

# Verify checksum against the stream metadata
EXPECTED_SHA=$(openshift-install coreos print-stream-json \
  | jq -r '.architectures.x86_64.artifacts.metal.formats.iso.disk["uncompressed"]')
echo "Expected SHA256: ${EXPECTED_SHA}"
sha256sum /var/www/html/ignition/scos9.iso
# Both SHA256 strings must match
```

Phase 2 — DNS Configuration (Windows Server)

All records live in the zone `ocp.svcexpert.net`. Run on the Windows DNS server as Administrator.

2.1 Forward Zone and A Records

```
powershell
```

```
# Create zone (skip if it already exists)
Add-DnsServerPrimaryZone -Name "ocp.svcexpert.net" `
    -ZoneFile "ocp.svcexpert.net.dns" -DynamicUpdate None

# A records
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "bastion" -IPv4Address 10.10.10.10
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "api" -IPv4Address 10.10.10.11
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "api-int" -IPv4Address 10.10.10.12
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "bootstrap" -IPv4Address 10.10.10.13
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "cp0" -IPv4Address 10.10.10.14
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "cp1" -IPv4Address 10.10.10.15
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "cp2" -IPv4Address 10.10.10.16
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "etcd-0" -IPv4Address 10.10.10.17
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "etcd-1" -IPv4Address 10.10.10.18
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "etcd-2" -IPv4Address 10.10.10.19

# Wildcard for apps (*.apps.ocp.svcexpert.net)
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "*" -IPv4Address 10.10.10.20
```

2.2 Reverse PTR Records

```
powershell

# Create reverse zone (skip if it already exists)
Add-DnsServerPrimaryZone -NetworkID "192.168.100.0/24" `
    -ZoneFile "100.168.192.in-addr.arpa.dns" -DynamicUpdate None

Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "10" -PtrRecord 10.10.10.10
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "11" -PtrRecord 10.10.10.11
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "12" -PtrRecord 10.10.10.12
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "14" -PtrRecord 10.10.10.14
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "15" -PtrRecord 10.10.10.15
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "16" -PtrRecord 10.10.10.16
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "17" -PtrRecord 10.10.10.17
```

2.3 etcd SRV Records

```
powershell
```

```

Add-DnsServerResourceRecord -ZoneName "ocp.svcexpert.net" -Srv `
  -Name "_etcd-server-ssl._tcp" -DomainName "etcd-0.ocp.svcexpert.net." `
  -Priority 0 -Weight 10 -Port 2380

Add-DnsServerResourceRecord -ZoneName "ocp.svcexpert.net" -Srv `
  -Name "_etcd-server-ssl._tcp" -DomainName "etcd-1.ocp.svcexpert.net." `
  -Priority 0 -Weight 10 -Port 2380

Add-DnsServerResourceRecord -ZoneName "ocp.svcexpert.net" -Srv `
  -Name "_etcd-server-ssl._tcp" -DomainName "etcd-2.ocp.svcexpert.net." `
  -Priority 0 -Weight 10 -Port 2380

# Verify all records
Get-DnsServerResourceRecord -ZoneName "ocp.svcexpert.net" | Format-Table -AutoS

```

2.4 DNS Validation from Bastion

All checks must pass before continuing. Do not proceed with any failure.

```

bash

DNS="192.168.100.251"

echo "=== Forward A Records ==="
dig @${DNS} api.ocp.svcexpert.net +short          # → 192.168.100.11
dig @${DNS} api-int.ocp.svcexpert.net +short      # → 192.168.100.12
dig @${DNS} bootstrap.ocp.svcexpert.net +short    # → 192.168.100.14
dig @${DNS} cp0.ocp.svcexpert.net +short          # → 192.168.100.15
dig @${DNS} cp1.ocp.svcexpert.net +short          # → 192.168.100.16
dig @${DNS} cp2.ocp.svcexpert.net +short          # → 192.168.100.17
dig @${DNS} console.apps.ocp.svcexpert.net +short # → 192.168.100.13

echo "=== Reverse PTR ==="
dig @${DNS} -x 192.168.100.15 +short              # → cp0.ocp.svcexpert.net.
dig @${DNS} -x 192.168.100.16 +short              # → cp1.ocp.svcexpert.net.
dig @${DNS} -x 192.168.100.17 +short              # → cp2.ocp.svcexpert.net.

echo "=== etcd SRV ==="
dig @${DNS} _etcd-server-ssl._tcp.ocp.svcexpert.net SRV +short
# Must return 3 lines referencing etcd-0, etcd-1, etcd-2

```

Phase 3 — Ignition File Generation

3.1 Generate SSH Key

```
bash

ssh-keygen -t ed25519 -f /root/.ssh/okd_id_ed25519 -C "okd-4.19" -N ""

# Display public key — paste into install-config.yaml below
cat /root/.ssh/okd_id_ed25519.pub
```

3.2 Create Minimal Pull Secret

OKD does not require a Red Hat subscription. This minimal secret is sufficient.

```
bash

cat > /opt/okd/pull-secret.json << 'EOF'
{"auths":{"fake":{"auth":"aWQ6cGFzcwo="}}}}
EOF
```

3.3 Create install-config.yaml

```
bash
```

```

mkdir -p /opt/okd/install

SSH_PUB_KEY=$(cat /root/.ssh/okd_id_ed25519.pub)

cat > /opt/okd/install/install-config.yaml << EOF
apiVersion: v1
baseDomain: svcexpert.net
metadata:
  name: ocp
compute:
  - hyperthreading: Enabled
    name: worker
    replicas: 0          # No workers initially
controlPlane:
  hyperthreading: Enabled
  name: master
  replicas: 3
networking:
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  machineNetwork:
    - cidr: 192.168.100.0/24
  networkType: OVNKubernetes
  serviceNetwork:
    - 172.30.0.0/16
platform:
  none: {}              # UPI - no platform automation
fips: false
pullSecret: '${(cat /opt/okd/pull-secret.json)}'
sshKey: '${SSH_PUB_KEY}'
EOF

# Review
cat /opt/okd/install/install-config.yaml

# CRITICAL: back up before generation consumes it
cp /opt/okd/install/install-config.yaml /opt/okd/install/install-config.yaml.backup

```

3.4 Generate Manifests and Ignition Files

```
bash
```

```
cd /opt/okd

# Step 1: Generate manifests
openshift-install create manifests --dir=/opt/okd/install/

# Step 2: Prevent masters from running workload pods
# (control-plane-only cluster until workers are added)
sed -i 's/mastersSchedulable: true/mastersSchedulable: false/' \
/opt/okd/install/manifests/cluster-scheduler-02-config.yml

grep mastersSchedulable /opt/okd/install/manifests/cluster-scheduler-02-config.yml
# Must show: mastersSchedulable: false

# Step 3: Generate ignition configs
openshift-install create ignition-configs --dir=/opt/okd/install/

# Verify all three ignition files exist
ls -lh /opt/okd/install/*.ign
# bootstrap.ign  master.ign  worker.ign
```

3.5 Publish Ignition Files via HTTPD

```
bash

cp /opt/okd/install/*.ign /var/www/html/ignition/
chmod 644 /var/www/html/ignition/*.ign
restorecon -Rv /var/www/html/ignition/

# Verify HTTP access and valid JSON
curl -s http://192.168.100.10:8080/ignition/bootstrap.ign | jq .ignition.version
curl -s http://192.168.100.10:8080/ignition/master.ign | jq .ignition.version
# Both must return a version string e.g. "3.4.0"
```

Ignition URLs (used in Phase 4 coreos-installer commands):

```
Bootstrap: http://192.168.100.10:8080/ignition/bootstrap.ign
Masters:   http://192.168.100.10:8080/ignition/master.ign
Workers:   http://192.168.100.10:8080/ignition/worker.ign
```

Phase 4 — VM Creation on ESXi 8 Console

SSH into the ESXi host for all commands in this phase.

4.1 Upload SCOS 9 ISO to ESXi Datastore

```
bash

# On ESXi host – download ISO from bastion's HTTPD
wget -O /vmfs/volumes/datastore1/scos9.iso \
    http://192.168.100.10:8080/ignition/scos9.iso

# Verify
ls -lh /vmfs/volumes/datastore1/scos9.iso
file /vmfs/volumes/datastore1/scos9.iso
# Must report: ISO 9660 ... (bootable)
```

4.2 VM Creation Script

Save this script on the ESXi host once, then call it for each VM.

```
bash
```

```
cat > /tmp/create-okd-vm.sh << 'VMSCRIPT'
#!/bin/sh
# Usage: create-okd-vm.sh <name> <cpus> <mem_mb> <disk_gb> [datastore] [portgroup]
VMNAME="$1"
CPUS="$2"
MEM="$3"
DISK_GB="$4"
DATASTORE="${5:-datastore1}"
PORTGROUP="${6:-VM Network}"
VMDIR="/vmfs/volumes/${DATASTORE}/${VMNAME}"
DISK_KB=$((DISK_GB * 1024 * 1024))

echo "=== Creating VM: ${VMNAME} ==="
mkdir -p "${VMDIR}"

# Create thin-provisioned VMDK
vmkfstools -c ${DISK_KB}k -d thin "${VMDIR}/${VMNAME}.vmdk"
echo "Disk created: ${VMDIR}/${VMNAME}.vmdk"

# Write VMX configuration
cat > "${VMDIR}/${VMNAME}.vmx" << VMXEOF
.encoding = "UTF-8"
config.version = "8"
virtualHW.version = "20"
displayName = "${VMNAME}"

# CPU
numvcpus = "${CPUS}"
cpuid.coresPerSocket = "${CPUS}"

# Memory
memSize = "${MEM}"

# Disk controller – pvscsi for best performance with CoreOS
scsi0.present = "TRUE"
scsi0.virtualDev = "pvscsi"
scsi0:0.present = "TRUE"
scsi0:0.fileName = "${VMNAME}.vmdk"
scsi0:0.deviceType = "scsi-hardDisk"

# CD-ROM – SCOS 9 live ISO
ide1:0.present = "TRUE"
ide1:0.fileName = "/vmfs/volumes/datastore1/scos9.iso"
```

```
ide1:0.deviceType = "cdrom-image"
ide1:0.startConnected = "TRUE"

# Network – vmxnet3 for best performance
ethernet0.present = "TRUE"
ethernet0.virtualDev = "vmxnet3"
ethernet0.networkName = "${PORTGROUP}"
ethernet0.addressType = "generated"
ethernet0.startConnected = "TRUE"

# Boot order – CD-ROM first (disk is blank until coreos-installer runs)
bios.bootOrder = "cdrom,hdd"
bios.bootDelay = "3000"

# Firmware – EFI without secure boot (required for SCOS)
firmware = "efi"
uefi.secureBoot.enabled = "FALSE"

# Guest OS
guestOS = "centos-64"

# Disable time sync (cluster manages its own time)
tools.syncTime = "FALSE"
tools.upgrade.policy = "manual"

# Required for CoreOS nested virt / CPU features
featCPU.vmx = "1"
vpmc.enable = "FALSE"
VMXEOF

# Register VM
VMID=$(vim-cmd solo/registervm "${VMDIR}/${VMNAME}.vmx")
echo "Registered '${VMNAME}' with ID: ${VMID}"
VMScript

chmod +x /tmp/create-okd-vm.sh
echo "Script ready."
```

4.3 Create All Four VMs

```
bash
```

```
# On ESXi host – create bootstrap and three control plane VMs
# Adjust "VM Network" to match your ESXi port group name

/tmp/create-okd-vm.sh bootstrap 4 16384 120 datastore1 "VM Network"
/tmp/create-okd-vm.sh cp0      4 16384 120 datastore1 "VM Network"
/tmp/create-okd-vm.sh cp1      4 16384 120 datastore1 "VM Network"
/tmp/create-okd-vm.sh cp2      4 16384 120 datastore1 "VM Network"

# Verify all four VMs are registered
vim-cmd vmsvc/getallvms
```

Note the numeric VM ID for each VM from the output — you will use these to power VMs on and off.

4.4 Node Installation Procedure (coreos-installer)

Repeat this procedure for each node: bootstrap, cp0, cp1, cp2.

Each VM boots the live SCOS 9 ISO into a live environment running in memory. You then configure a static IP with `nmcli`, verify connectivity to the bastion, run `coreos-installer install` to write SCOS 9 to disk with the correct ignition file, disconnect the CD-ROM, and reboot.

General Steps (do this for each VM)

Step 1: Power on the VM from the ESXi host:

```
bash

# On ESXi host – replace N with the VM's numeric ID from vim-cmd vmsvc/getallvms
vim-cmd vmsvc/power.on N
```

Step 2: Open the VM console in the ESXi web UI. Wait for the SCOS live environment to boot to an automatic login prompt (the `core` user, no password). This takes 1-3 minutes.

Step 3: Check the NIC name in the live environment:

```
bash

nmcli device status
# Usually: ens192 on VMware (vmxnet3 adapter)
# Note the actual name – substitute it below if different
```

Step 4: Configure static IP (commands differ per node — see below).

Step 5: Verify connectivity:

```
bash

ip addr show
ping -c 3 192.168.100.10 # Must reach ba
curl -s http://192.168.100.10:8080/ignition/bootstrap.ign | head -c 50
# Must return the start of JSON: {"ignition":{"version":...
```

Step 6: Run coreos-installer (command differs per node — see below).

Step 7: Before rebooting — disconnect the CD-ROM in the ESXi web UI so the VM boots from disk on the next start:

- Select the VM → Edit Settings → CD/DVD Drive 1 → uncheck "Connected"
- Click Save

Step 8: Reboot:

```
bash

sudo systemctl reboot
```

Bootstrap VM (192.168.100.14)

```
bash
```

```
# === Run at the bootstrap VM console ===

# Step 3: Check NIC name
nmcli device status

# Step 4: Configure static IP
sudo nmcli con mod "Wired connection 1" \
  ipv4.addresses "192.168.100.14/24" \
  ipv4.gateway "192.168.100.1" \
  ipv4.dns "192.168.100.251" \
  ipv4.method manual \
  connection.id "ens192"
sudo nmcli con up "Wired connection 1"

# Step 5: Verify
ip addr show
ping -c 3 192.168.100.10
curl -s http://192.168.100.10:8080/ignition/bootstrap.ign | head -c 50

# Step 6: Install SCOS 9 to disk with bootstrap ignition
sudo coreos-installer install /dev/sda \
  --ignition-url http://192.168.100.10:8080/ignition/bootstrap.ign \
  --copy-network \
  --insecure-ignition

# Confirm success message: "Install complete."
# Step 7: Disconnect CD-ROM in ESXi web UI first, then:
sudo systemctl reboot
```

Control Plane 0 — cp0 (192.168.100.15)

```
bash
```

```
# === Run at the cp0 VM console ===
```

```
sudo nmcli con mod "Wired connection 1" \  
  ipv4.addresses "192.168.100.15/24" \  
  ipv4.gateway   "192.168.100.1" \  
  ipv4.dns       "192.168.100.251" \  
  ipv4.method    manual \  
  connection.id  "ens192"  
sudo nmcli con up "Wired connection 1"
```

```
ping -c 3 192.168.100.10
```

```
curl -s http://192.168.100.10:8080/ignition/master.ign | head -c 50
```

```
sudo coreos-installer install /dev/sda \  
  --ignition-url http://192.168.100.10:8080/ignition/master.ign \  
  --copy-network \  
  --insecure-ignition
```

```
# Disconnect CD-ROM in ESXi web UI, then:
```

```
sudo systemctl reboot
```

Control Plane 1 — cp1 (192.168.100.16)

```
bash
```

```
# === Run at the cp1 VM console ===
```

```
sudo nmcli con mod "Wired connection 1" \  
  ipv4.addresses "192.168.100.16/24" \  
  ipv4.gateway   "192.168.100.1" \  
  ipv4.dns       "192.168.100.251" \  
  ipv4.method    manual \  
  connection.id  "ens192"  
sudo nmcli con up "Wired connection 1"
```

```
ping -c 3 192.168.100.10
```

```
curl -s http://192.168.100.10:8080/ignition/master.ign | head -c 50
```

```
sudo coreos-installer install /dev/sda \  
  --ignition-url http://192.168.100.10:8080/ignition/master.ign \  
  --copy-network \  
  --insecure-ignition
```

```
# Disconnect CD-ROM in ESXi web UI, then:
```

```
sudo systemctl reboot
```

Control Plane 2 — cp2 (192.168.100.17)

```
bash
```

```
# === Run at the cp2 VM console ===

sudo nmcli con mod "Wired connection 1" \
  ipv4.addresses "192.168.100.17/24" \
  ipv4.gateway   "192.168.100.1" \
  ipv4.dns       "192.168.100.251" \
  ipv4.method    manual \
  connection.id  "ens192"
sudo nmcli con up "Wired connection 1"

ping -c 3 192.168.100.10
curl -s http://192.168.100.10:8080/ignition/master.ign | head -c 50

sudo coreos-installer install /dev/sda \
  --ignition-url http://192.168.100.10:8080/ignition/master.ign \
  --copy-network \
  --insecure-ignition

# Disconnect CD-ROM in ESXi web UI, then:
sudo systemctl reboot
```

Phase 5 — Sequential Power-On and Deployment

Follow this order exactly. Do not power on the next node until all verification checks for the current step pass.

Pre-flight: Verify Bastion Services

Run this before powering on any node.

```
bash
```

```
# 1. All four IPs active on ens18
ip addr show ens18 | grep "inet "
# Must show .10, .11, .12, .13

# 2. HAProxy running and listening
systemctl is-active haproxy
ss -tlnp | grep -E ':6443|:22623|:9000'
curl -s http://127.0.0.1:9000/stats | grep -c "FRONTEND"

# 3. Ignition files served over HTTP
curl -s http://192.168.100.10:8080/ignition/bootstrap.ign | jq .ignition.version
curl -s http://192.168.100.10:8080/ignition/master.ign | jq .ignition.version
# Both must return a version string

# 4. DNS resolves correctly from bastion
dig @192.168.100.251 api.ocp.svcexpert.net +short # → 192.168.100.11
dig @192.168.100.251 cp0.ocp.svcexpert.net +short # → 192.168.100.15
```

STEP 1 — Bootstrap

```
bash

# On ESXi host
BOOTSTRAP_ID=$(vim-cmd vmvc/getallvms | grep bootstrap | awk '{print $1}')
vim-cmd vmvc/power.on ${BOOTSTRAP_ID}
```

At the bootstrap console: run the bootstrap `coreos-installer` procedure from Phase 4.4.
After reboot, wait for the bootstrap API to respond:

```
bash
```

```
# On bastion – poll until bootstrap API is reachable (~5-15 min after reboot)
echo "Waiting for bootstrap API..."
until curl -k -s https://api.ocp.svcexpert.net:6443/version > /dev/null 2>&1; do
    printf "."
    sleep 15
done
echo ""
echo "Bootstrap API is up!"

# Verify bootstrap services are running
ssh -i /root/.ssh/okd_id_ed25519 core@192.168.100.14
# On bootstrap node:
systemctl status release-image.service --no-pager
systemctl status bootkube.service --no-pager
journalctl -b -f -u bootkube.service
# Exit with Ctrl+C when you see etcd-related log output
```

STEP 2 — Control Plane Node 0 (cp0)

```
bash

# On ESXi host
CP0_ID=$(vim-cmd vmshvc/getallvms | grep "^[0-9]" | grep " cp0 " | awk '{print $1}')
vim-cmd vmshvc/power.on ${CP0_ID}
```

At the cp0 console: run the cp0 `coreos-installer` procedure from Phase 4.4.

Wait for cp0 to contact bootstrap (~3-5 min after reboot):

```
bash

# On bastion
export KUBECONFIG=/opt/okd/install/auth/kubeconfig
oc get nodes
# cp0 may appear as NotReady initially – that is normal at this stage
```

STEP 3 — Control Plane Node 1 (cp1)

```
bash
```

```
CP1_ID=$(vim-cmd vmshvc/getallvms | grep "^[0-9]" | grep " cp1 " | awk '{print $1}')
vim-cmd vmshvc/power.on ${CP1_ID}
```

At the cp1 console: run the cp1 `coreos-installer` procedure.

```
bash

# Monitor etcd membership forming
export KUBECONFIG=/opt/okd/install/auth/kubeconfig
oc get etcd cluster -o jsonpath='{.status.conditions[?(@.type=="EtcdMembersAvailable")].status}'
# Will eventually return: True
```

STEP 4 — Control Plane Node 2 (cp2)

```
bash

CP2_ID=$(vim-cmd vmshvc/getallvms | grep "^[0-9]" | grep " cp2 " | awk '{print $1}')
vim-cmd vmshvc/power.on ${CP2_ID}
```

At the cp2 console: run the cp2 `coreos-installer` procedure.

STEP 5 — Wait for Bootstrap Complete

This command blocks until it is safe to remove bootstrap. Typical wait: 20-45 minutes from when all three CPs are up.

```
bash

# On bastion — do NOT interrupt this
openshift-install --dir=/opt/okd/install/ wait-for bootstrap-complete \
  --log-level=info 2>&1 | tee /opt/okd/bootstrap-complete.log

# Success output:
# INFO Bootstrap status: complete
# INFO It is now safe to remove the bootstrap resources
```

While waiting, monitor in another terminal:

```
bash
```

```
export KUBECONFIG=/opt/okd/install/auth/kubeconfig

# Watch nodes coming up
watch -n 10 "oc get nodes"

# Watch cluster operators
watch -n 15 "oc get clusteroperators"

# Watch etcd
oc get etcd cluster -o jsonpath='{.status.conditions}' | jq .
```

Phase 6 — Bootstrap Removal and Cluster Completion

6.1 Remove Bootstrap from HAProxy

```
bash

# On bastion - comment out bootstrap from both backends
sed -i 's/^    server bootstrap 192.168.100.14:6443.*$/    #server bootstrap 192.168.100.14:6443.*$/' /etc/haproxy/haproxy.cfg
sed -i 's/^    server bootstrap 192.168.100.14:22623.*$/    #server bootstrap 192.168.100.14:22623.*$/' /etc/haproxy/haproxy.cfg

# Graceful reload (no downtime)
systemctl reload haproxy

# Verify bootstrap is gone from active backends
curl -s http://127.0.0.1:9000/stats | grep -i bootstrap
```

6.2 Power Off and Delete Bootstrap VM

```
bash
```

```
# On ESXi host
BOOTSTRAP_ID=$(vim-cmd vmvc/getallvms | grep bootstrap | awk '{print $1}')

vim-cmd vmvc/power.off ${BOOTSTRAP_ID}
sleep 15

vim-cmd vmvc/unregister ${BOOTSTRAP_ID}
rm -rf /vmfs/volumes/datastore1/bootstrap/

echo "Bootstrap VM deleted."
```

6.3 Wait for All Cluster Operators

```
bash

# On bastion
openshift-install --dir=/opt/okd/install/ wait-for install-complete \
  --log-level=info 2>&1 | tee /opt/okd/install-complete.log

# Success output:
# INFO Install complete!
# INFO Access the OpenShift web-console here: https://console.apps.ocp.svcexper
# INFO Login with kubectl: <password>
```

6.4 Retrieve Credentials

```
bash

# CLI access
export KUBECONFIG=/opt/okd/install/auth/kubeconfig
oc whoami    # → system:admin

# Web console password
cat /opt/okd/install/auth/kubeadmin-password

echo "Web Console: https://console.apps.ocp.svcexpert.net"
echo "Username:    kubeadmin"
echo "Password:    $(cat /opt/okd/install/auth/kubeadmin-password)"
```

Store `kubeadmin-password` and `/opt/okd/install/auth/kubeconfig` in a safe location. These are your only initial cluster admin credentials.

6.5 Final Cluster Health Check

```
bash

export KUBECONFIG=/opt/okd/install/auth/kubeconfig

# All three nodes must be Ready
oc get nodes -o wide

# All operators must show Available=True, Progressing=False, Degraded=False
oc get clusteroperators

# Cluster version must be non-progressing
oc get clusterversion

# Key system pod health
oc get pods -n openshift-etcd | grep -v Running
oc get pods -n openshift-kube-apiserver | grep -v Running
oc get pods -n openshift-kube-scheduler | grep -v Running

# API reachable externally
curl -k https://api.ocp.svcexpert.net:6443/version

# Console reachable
curl -k -I https://console.apps.ocp.svcexpert.net
```

Phase 7 — Worker Node Addition (Future)

Complete Phase 6 verification fully before starting this phase. All cluster operators must show `Available: True`.

7.1 Add DNS Records for Workers (Windows DNS)

```
powershell

Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "worker0" -IPv
Add-DnsServerResourceRecordA -ZoneName "ocp.svcexpert.net" -Name "worker1" -IPv
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "20"
Add-DnsServerResourceRecordPtr -ZoneName "100.168.192.in-addr.arpa" -Name "21"
```

7.2 Update HAProxy for Ingress Traffic

```
bash

# On bastion – edit /etc/haproxy/haproxy.cfg
# In ingress-http-backend and ingress-https-backend, uncomment:
#   server worker0 192.168.100.20:80 check
#   server worker1 192.168.100.21:80 check
# (and the corresponding :443 lines)

vi /etc/haproxy/haproxy.cfg
systemctl reload haproxy
```

7.3 Create Worker VMs

```
bash

# On ESXi host
/tmp/create-okd-vm.sh worker0 4 16384 120 datastore1 "VM Network"
/tmp/create-okd-vm.sh worker1 4 16384 120 datastore1 "VM Network"
```

7.4 Install Workers with coreos-installer

Power on each worker VM, then at its console:

```
bash
```

```
# === worker0 console (192.168.100.20) ===
sudo nmcli con mod "Wired connection 1" \
  ipv4.addresses "192.168.100.20/24" \
  ipv4.gateway "192.168.100.1" \
  ipv4.dns "192.168.100.251" \
  ipv4.method manual \
  connection.id "ens192"
sudo nmcli con up "Wired connection 1"

ping -c 3 192.168.100.10
curl -s http://192.168.100.10:8080/ignition/worker.ign | head -c 50

sudo coreos-installer install /dev/sda \
  --ignition-url http://192.168.100.10:8080/ignition/worker.ign \
  --copy-network \
  --insecure-ignition

# Disconnect CD-ROM in ESXi web UI, then:
sudo systemctl reboot
```

Repeat for `worker1` (192.168.100.21).

7.5 Approve Worker Certificate Signing Requests

Workers don't auto-join. You must approve their CSRs twice — once for the bootstrapper cert, once for the serving cert.

```
bash

export KUBECONFIG=/opt/okd/install/auth/kubeconfig

# Watch for pending CSRs (run after workers boot)
watch -n 10 "oc get csr | grep Pending"

# First approval round (bootstrap CSRs)
oc get csr -o name | xargs oc adm certificate approve

# Wait 1-2 minutes, then second approval round (serving CSRs)
oc get csr -o name | xargs oc adm certificate approve

# Workers should now appear
oc get nodes
```

Appendix — Troubleshooting

Quick Cluster Health Script

bash

```
cat > /opt/okd/check-cluster.sh << 'EOF'
#!/bin/bash
export KUBECONFIG=/opt/okd/install/auth/kubeconfig
echo "=== Nodes ==="
oc get nodes -o wide
echo ""
echo "=== Cluster Operators (non-healthy) ==="
oc get co | grep -v "True.*False.*False" || echo "All operators healthy"
echo ""
echo "=== Cluster Version ==="
oc get clusterversion
echo ""
echo "=== etcd Status ==="
oc get etcd cluster -o jsonpath='{.status.conditions[?(@.type=="EtcdMembersAvailable")].message}'
echo ""
echo "=== HAProxy Backends ==="
curl -s http://127.0.0.1:9000/stats | grep -E "BACKEND|UP|DOWN" | head -20
EOF
chmod +x /opt/okd/check-cluster.sh
```

Problem: coreos-installer NIC Name Differs from "Wired connection 1"

bash

```
# At live console – list all connection profiles
nmcli con show

# List devices and their current connection profile names
nmcli device status

# If the connection profile is named differently (e.g. "ens192" directly):
sudo nmcli con mod "ens192" \
    ipv4.addresses "192.168.100.15/24" \
    ipv4.gateway "192.168.100.1" \
    ipv4.dns "192.168.100.251" \
    ipv4.method manual
sudo nmcli con up "ens192"
```

Problem: VM Boots Back to Live ISO After coreos-installer

The CD-ROM was not disconnected before the reboot.

```
bash

# On ESXi host – power off the VM first
vim-cmd vmsvc/power.off <VM_ID>

# Then in the ESXi web UI:
# Select VM → Edit Settings → CD/DVD Drive 1 → uncheck Connected → Save

# Then power on again
vim-cmd vmsvc/power.on <VM_ID>
# The VM now boots from /dev/sda (the installed disk)
```

Alternatively, set the VMX directly on the ESXi host:

```
bash

# On ESXi host
VMX="/vmfs/volumes/datastore1/<vmname>/<vmname>.vmx"
sed -i 's/ide1:0.startConnected = "TRUE"/ide1:0.startConnected = "FALSE"/' "$VMX"
# Reload the VM config
vim-cmd vmsvc/reload <VM_ID>
vim-cmd vmsvc/power.on <VM_ID>
```

Problem: Bootstrap API Not Responding After Install

```
bash
```

```
# Verify bootstrap booted from disk (not live ISO again)
```

```
ping -c 3 192.168.100.14
```

```
# Test port 6443 directly on bootstrap (bypasses HAProxy)
```

```
timeout 3 bash -c "echo > /dev/tcp/192.168.100.14/6443" && echo OPEN || echo CL
```

```
# SSH to bootstrap and check bootkube progress
```

```
ssh -i /root/.ssh/okd_id_ed25519 core@192.168.100.14
```

```
systemctl status bootkube.service --no-pager
```

```
journalctl -b -f -u release-image.service -u bootkube.service
```

```
# Look for: image pulls completing, etcd starting, API server starting
```

```
# Check ignition was applied (network should be static, not DHCP)
```

```
ip addr show
```

```
cat /etc/hostname
```

```
# Must show: bootstrap.ocp.svcexpert.net
```

Problem: Control Plane Not Joining

```
bash
```

```
# SSH to control plane node
```

```
ssh -i /root/.ssh/okd_id_ed25519 core@192.168.100.15
```

```
# Check kubelet
```

```
systemctl status kubelet --no-pager
```

```
journalctl -b -f -u kubelet
```

```
# Check if MCS is reachable from the node
```

```
curl -k https://api-int.ocp.svcexpert.net:22623/config/master
```

```
# Check DNS resolves from inside the node
```

```
cat /etc/resolv.conf
```

```
nslookup api-int.ocp.svcexpert.net
```

Problem: etcd Not Forming Quorum

bash

```
export KUBECONFIG=/opt/okd/install/auth/kubeconfig
```

```
# Check etcd pod status
```

```
oc get pods -n openshift-etcd -o wide
```

```
# Check etcd operator
```

```
oc logs -n openshift-etcd-operator deployment/etcd-operator --tail=50
```

```
# Detailed etcd condition
```

```
oc get etcd cluster -o yaml | grep -A 30 "conditions:"
```

```
# Check for clock skew between nodes (etcd is sensitive to time drift)
```

```
for IP in 192.168.100.15 192.168.100.16 192.168.100.17; do
```

```
    echo -n "${IP}: "
```

```
    ssh -i /root/.ssh/okd_id_ed25519 core@${IP} "date" 2>/dev/null
```

```
done
```

Problem: Cluster Operators Stuck Progressing

bash

```
export KUBECONFIG=/opt/okd/install/auth/kubeconfig

# Find which operators are not healthy
oc get co | grep -v "True.*False.*False"

# Get detailed reason for a specific operator (e.g. dns)
oc describe co dns | grep -A 10 "Message:"

# Check operator pod logs
oc get pods -n openshift-dns -o wide
oc logs -n openshift-dns -l dns.operator.openshift.io/daemonset-dns --tail=50

# Force a re-sync if an operator is stuck
oc patch <operator-name> cluster \
  --type=merge \
  -p '{"spec":{"managementState":"Unmanaged"}}'
# Wait 30s then set back to Managed
oc patch <operator-name> cluster \
  --type=merge \
  -p '{"spec":{"managementState":"Managed"}}'
```

Problem: HAProxy Health Check Failures

```
bash

# View live backend status in stats page
curl -s http://127.0.0.1:9000/stats | grep -E "cp[0-9]|bootstrap"

# Test connectivity to each control plane API port directly
for IP in 192.168.100.15 192.168.100.16 192.168.100.17; do
  echo -n "${IP}:6443 → "
  timeout 3 bash -c "echo > /dev/tcp/${IP}/6443" 2>/dev/null \
    && echo "OPEN" || echo "CLOSED"
  echo -n "${IP}:22623 → "
  timeout 3 bash -c "echo > /dev/tcp/${IP}/22623" 2>/dev/null \
    && echo "OPEN" || echo "CLOSED"
done

# Check HAProxy error log
journalctl -u haproxy --since "1 hour ago" | tail -20
```

Problem: `node-image-pull.service` Fails (If Using OKD < 4.19)

This error: `Expected single docker ref, found: ... ostree/container/image/...` **does not occur on OKD 4.19+** when using the correct SCOS 9 ISO obtained via `openshift-install coreos print-stream-json`. If you see this error, confirm your OKD_VERSION is exactly `4.19.0-okd-scos.3` and that you downloaded the ISO using the `print-stream-json` command, not a manually guessed URL.

Post-Install Checklist

```
[ ] All 3 control plane nodes show STATUS=Ready:
    oc get nodes
[ ] All cluster operators Available=True, Degraded=False:
    oc get co
[ ] Cluster version not progressing:
    oc get clusterversion
[ ] Web console accessible:
    https://console.apps.ocp.svcexpert.net
[ ] kubeadmin password saved securely:
    /opt/okd/install/auth/kubeadmin-password
[ ] kubeconfig backed up:
    /opt/okd/install/auth/kubeconfig
[ ] install-config backup saved:
    /opt/okd/install-config.yaml.backup
[ ] SSH key backed up:
    /root/.ssh/okd_id_ed25519
[ ] Bootstrap VM deleted from ESXi
[ ] Bootstrap removed from HAProxy config
```

Guide: OKD 4.19.0-okd-scos.3 / SCOS 9 / VMware ESXi 8 (No vCenter) / UPI / Rocky Linux 9 Bastion